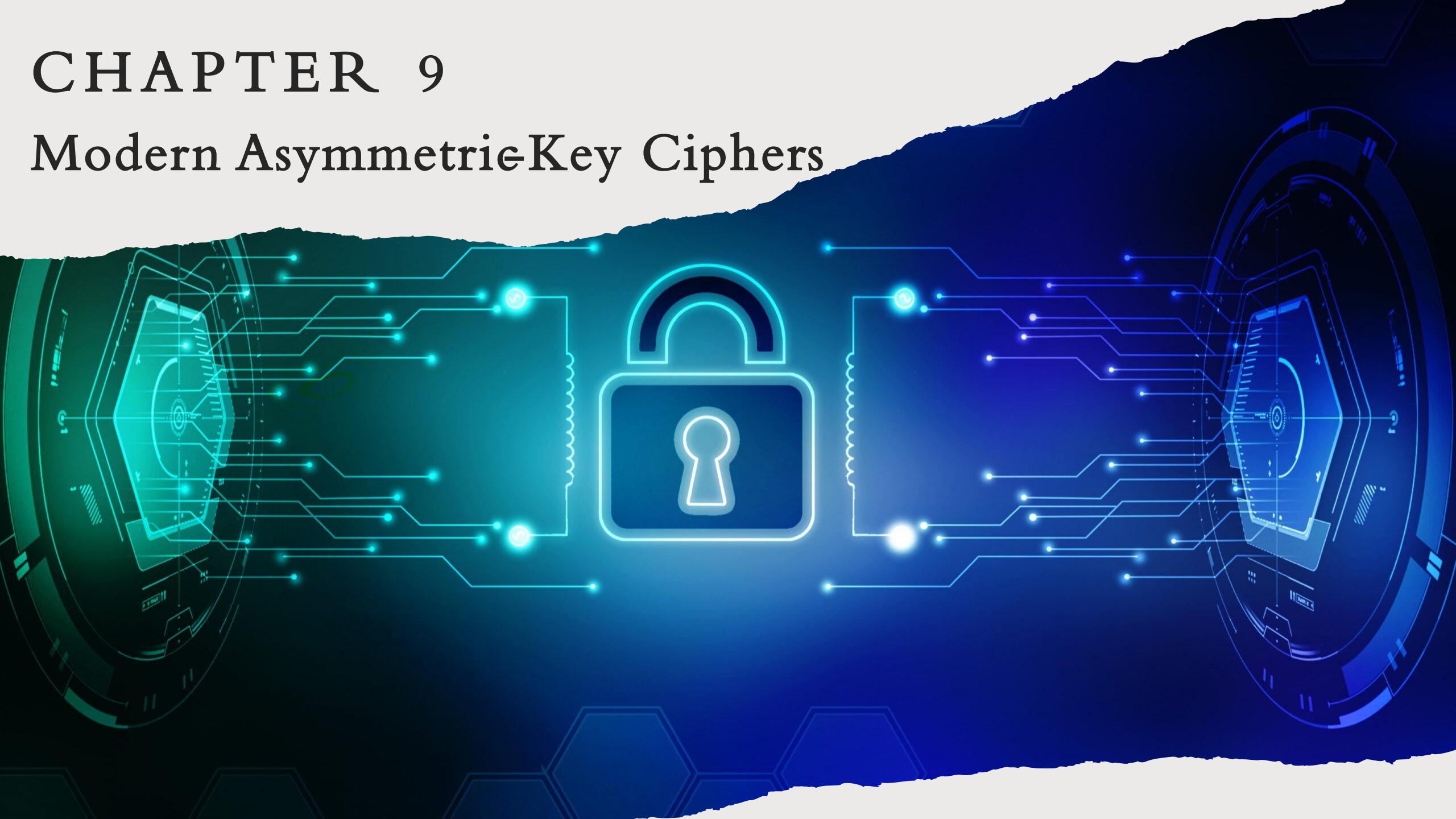
CHAPTER 9

Modern Asymmetric-Key Ciphers



OUTLINES

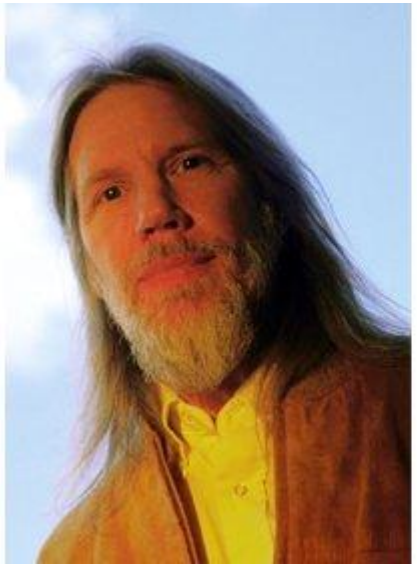
1. Diffie-Hellman Key Exchange.
2. Knapsack Cryptosystem.
3. RSA Cryptosystem.
4. ElGamal Cryptosystem (optional).

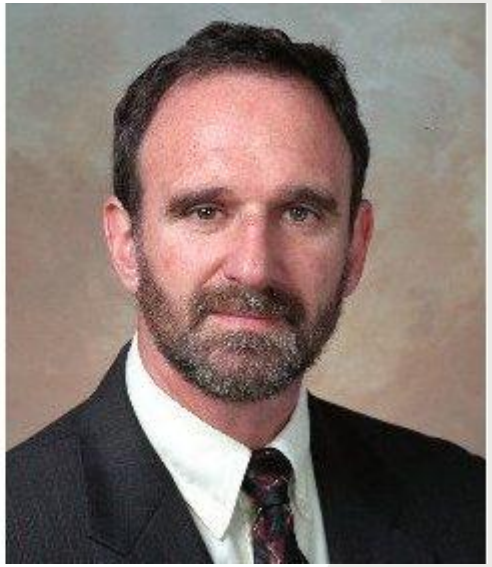
LEARNING OBJECTIVES

In the end of this Chapter 9, students will be able:

1. To understand and operate the **Diffie-Hellman Key Exchange** to establish a commonshared secretkey.
2. To understand and operate the **knapsack cryptosystem** to encrypt and decryptmessages.
3. To understand and operate the **RSA cryptosystem** to encrypt and decryptmessages.
4. To understand and discuss some **security issues** in the above-mentioned asymmetric-key cryptography

DIFFIE -HELLMAN KEY EXCHANGE





DIFFIE-HELLMAN KEY EXCHANGE (DHKE)

- The birth of **first ever public-key cryptography** in 1976, thanks to Whitfield Diffie, Martin Hellman and Ralph Merkle.
- **DHKE** resolves the **key distribution issue** addressed in modern cryptography.
  - ✓ It enables two users to **securely exchange a secret key** that can be used for subsequent symmetric encryption of messages.
  - ✓ This algorithm limited to the exchange of secret key (**no encryption** is considered)
  - ✓ The underlying security lies on the **difficulty assumption of computing the Discrete Logarithm Problem (DLP)** .

DHKE – THE DLP ASSUMPTION

❖Definition 1:

Let g be a primitive root for finite field $\mathbb{F}_q$ of prime q and let h be a non-zero element of $\mathbb{F}_q$. The DLP is the problem of finding an integer (exponent) x such that $h \equiv g^x \pmod{p}$.

❖In other words, this integer x is the discrete logarithm of h to the base g.

❖The assumption is made such that finding this exponent x is ***computationally infeasible*** for a large finite field.

DHKE – ALGORITHM

Public Parameter Generation	
A Trusted Third Party (TTP) chooses and publishes a large prime p and an element $g \pmod p$ of large prime order.	
Alice	**Bob**
Public Key Generation	
1. Chooses private key $1 \leq a \leq p-1$. 2. Computes $A \equiv g^a \pmod p$. 3. Publishes her public key A to Bob.	1. Chooses private key $1 \leq b \leq p-1$. 2. Computes $B \equiv g^b \pmod p$. 3. Publishes his public key B to Alice.
Secret Key Generation $g^{ab} mod\ p$ = Symmetric key used for encryption	
1. Computes $B^a \pmod p$.	1. Computes $A^b \pmod p$.

- The shared private key computed is **correct** since:

$$B^a \pmod p \equiv \left(g^b\right)^a \equiv (g^a)^b \equiv A^b \pmod p$$

DHKE – AN EXAMPLE

Public Parameter Generation	
A Trusted Third Party (TTP) chooses and publishes a large prime $p = 353$ and an element $g = 3$ of large prime order.	
Alice	**Bob**
Public Key Generation	
1. Chooses private key $a = 97$. 2. Computes $A \equiv 3^{97} \equiv 40 \pmod{353}$. 3. Publishes her public key $A \equiv 40 \pmod{353}$.	1. Chooses private key $b = 233$. 2. Computes $B \equiv 3^{233} \equiv 248 \pmod{353}$. 3. Publishes his public key $B \equiv 248 \pmod{353}$.
Secret Key Generation (symmetric key for encryption)	
1. Computes $248^{97} \equiv \mathbf{160} \pmod{\mathbf{353}}$.	1. Computes $40^{233} \equiv \mathbf{160} \pmod{\mathbf{353}}$.

- If an attacker captures the public key A or B, he needs to **find out the private keys a or b that generates them**. This is indeed the DLP.

$$248^{97} \equiv 40^{233}$$
$$(3)^{(233)97} \equiv (3)^{(97)(233)}$$

139751

DHKE – SECURITY

- **Security** behind Diffie-Hellman Key Exchange:
  1. (Strongest) Discrete Logarithm Problem (DLP): Given X, g, p : $X \equiv g^x \pmod p$, find the discrete logarithm x. Given p = 323; g = 3, X = 40. Find x = ?.
  2. Computational Diffie-Hellman Problem (CDH): Given $X \equiv g^x \pmod p$ and $Y \equiv g^y \pmod p$, computes $g^{xy} \pmod p$. Given public keys from Alice and Bob, compute their share secret. All Public key are known by everybody. Having known the public key, find shared secret.
  3. Decisional Diffie-Hellman Problem (DDH): Given 2 sets of $\{g, g^x, g^y, g^{xy}\}$ and $\{g, g^x, g^y, g^z\}$, decide whether $g^{xy} \equiv g^z \pmod p$. You found g^z but unsure that $g^z = g^{xy}$

- DLP is one of the **NP problems** but NOT **NP-Complete**
  - ✓NOT NP-Complete as DLP has no reduction into other NP-Complete problems

DHKE – SECURITY ISSUE

- Although the idea is brilliant, unfortunately the basic version of DHKE suffers from the **Man-in-the-Middle (MITM) attack**.
- **Reason**: DHKE protocol **DOES NOT authenticate** the users.
- **Solution**: The MITM in DHKE can be resolved using additional **digital signature** and **public-key certificates**.

- In the next few slides, the demonstration on how MITM is possible in DHKE is outlined.

DHKE – MAN-IN-THE-MIDDLE ATTACK

- Suppose Alice and Bob wish to exchange keys, and **Eve is the adversary**. The attack proceedsas follows.

1. Eve prepares for the attack by generating two random private keys, e_1 and e_2, and then computingthe correspondingpublic keys, E_1 and E_2.
2. Alice transmits A to Bob.
3. **Eve intercepts *A***, transmits E_1 to Bob, and computes $K_2 \equiv A^{e_2} \pmod{p}$.
4. Bob receives E_1 and calculates $K_1 \equiv E_1^b \pmod{p}$.
5. Bob transmits B to Alice.
6. **Eve intercepts *B***, transmits E_2 to Alice, and computes $K_1 \equiv B^{e_1} \pmod{p}$.
7. Alice receives E_2 and calculates $K_2 \equiv E_2^a \pmod{p}$.

Eve pretend to be Bob for Alice.
Eve pretend to be Alice for Bob.
Eve does the DHKE 2x.

DHKE – MAN-IN-THE-MIDDLE ATTACK

Public Parameter Generation		
A Trusted Third Party (TTP) chooses and publishes a large prime p and an element $g \pmod p$ of large prime order.		
Alice	**Eve**	**Bob**
Public Key Generation		
1. Chooses private key $1 \leq a \leq p-1$. 2. Computes $A \equiv g^a \pmod p$. 3. Publishes her public key A to Bob.	1. Chooses private keys $1 \leq e_1, e_2 \leq p-1$ 2. Computes $E_1 \equiv g^{e_1} \pmod p$ and $E_2 \equiv g^{e_2} \pmod p$. 3. Captures B and sends E_1 to Bob. 4. Captures A and sends E_2 to Alice.	1. Chooses private key $1 \leq b \leq p-1$. 2. Computes $B \equiv g^b \pmod p$. 3. Publishes his public key B to Alice.
Secret Key Generation		
1. Computes $K_2 \equiv E_2^a \equiv g^{ae_2} \pmod p$.	1. Computes $K_1 \equiv B^{e_1} \equiv g^{be_1} \pmod p$. 2. Computes $K_2 \equiv A^{e_2} \equiv g^{ae_2} \pmod p$.	1. Computes $K_1 \equiv E_1^b \equiv g^{be_1} \pmod p$.

DHKE – MAN-IN-THE-MIDDLE ATTACK

- At this point, Bob and Alice **think** that they share a secret key. Instead, **Bob and Eve share secret key K_1**, and **Alice and Eve share secret key K_2**. All future communication between Bob and Alice is compromised in the following way:

1. Alice sends an encrypted message M, $\mathrm{Enc}(K_2, M)$.
2. Eve intercepts the encrypted message and decrypts it, to recover M.
3. Eve sends Bob $\mathrm{Enc}(K_1, M)$ or $\mathrm{Enc}(K_2, M')$, where M' is any message. In the first case, Eve wants to eavesdrop on the communication without altering it. In the second case, Eve wants to modify the message going to Bob.

KNAPSACK
CRYPTOSYSTEM

KNAPSACK CRYPTOSYSTEM

- One of the earliest **public-key cryptosystems for encryption** in 1978 proposed by Ralph Merkle and Martin Hellman after the birth of DHKE.
- The fundamental primitive of Knapsack cryptosystem is based on the **0/1 knapsack problem**, an example of the combinational optimization problem.

KNAPSACK PROBLEM

❖ **Definition 2**:

Given N items where each has some weights w_i and value v_i associated with it. Given a knapsack with capacity W. The target is to **maximize the items** into the knapsack such that **the sum of values associated with them is the maximum possible**.

➢ Mathematically,

$$\sum_{i=1}^{N} v_i x_i \leq W, \qquad \text{where } x_i \in \{0,1\}$$

KNAPSACK FUNCTION – SETUP

- Based on **Definition 2**, the design of knapsack cryptosystem is defined in the following setup:

1. Given two k-tuples, $\mathbf{a} = [a_1, a_2, \ldots, a_k]$ and $\mathbf{x} = [x_1, x_2, \ldots, x_k]$ with the tuple $\mathbf{a}$ be the pre-defined set and $\mathbf{x} \in \{0,1\}$.
2. The sum of the elements in the knapsack is

$$s = \text{knapsackSum}(\mathbf{a}, \mathbf{x}) = a_1x_1 + a_2x_2 + \cdots + a_kx_k$$

- The $\text{knapsackSum}(\mathbf{a}, \mathbf{x})$ is a **one-way function** if the k-tuples $\mathbf{a}$ is a general tuple (**public parameter**).

KNAPSACK FUN $\mathbf{a} = [a_1, a_2, \ldots, a_k]$ AND $\mathbf{x} = [x_1, x_2, \ldots, x_k]$ CTION & INVERSE

U have S, a, k
Find x

```
knapsackSum (x [1 … k], a [1 … k])
{
    s ← 0
    for (i = 1 to k)
    {
      s ← s + a_i × x_i
    }
    return s
}
```

U have x, a, k
Find S

```
inv_knapsackSum (s, a [1 … k])
{
    for (i = k down to 1)
    {
        if s ≥ a_i
        {
            x_i ← 1
            s ← s − a_i
        }
        else x_i ← 0
    }
    return x [1 … k]
}
```

$\mathbf{a} = [a_1, a_2, \ldots, a_k]$ and $\mathbf{x} = [x_1, x_2, \ldots, x_k]$

$\mathbf{a} = [a_1, a_2, \ldots, a_k]$ and $\mathbf{x} = [x_1, x_2, \ldots, x_k]$

KNAPSACK FUNCTION & INVERSE

- For the knapsack function to be invertible, the required trapdoor is that the general k-tuples $\mathbf{a} = [a_1, a_2, \ldots, a_k]$ is a **super-increasing** tuple, such that:

$$a_i \geq a_1 + a_2 + \cdots + a_{i-1}$$

$\{2, 3, 7, 13, 30\}$: $7 > 2+3$; $13 > 2+3+7$; ….

- In other words, each element (except a_1) is **greater than or equal to** the **sum of all previous elements**.

$\mathbf{a} = [17,25,46,94,201,400]$

x = 0 ,1, 1,0, 1, 0

KNAPSACK FUNCTION & INVERSE

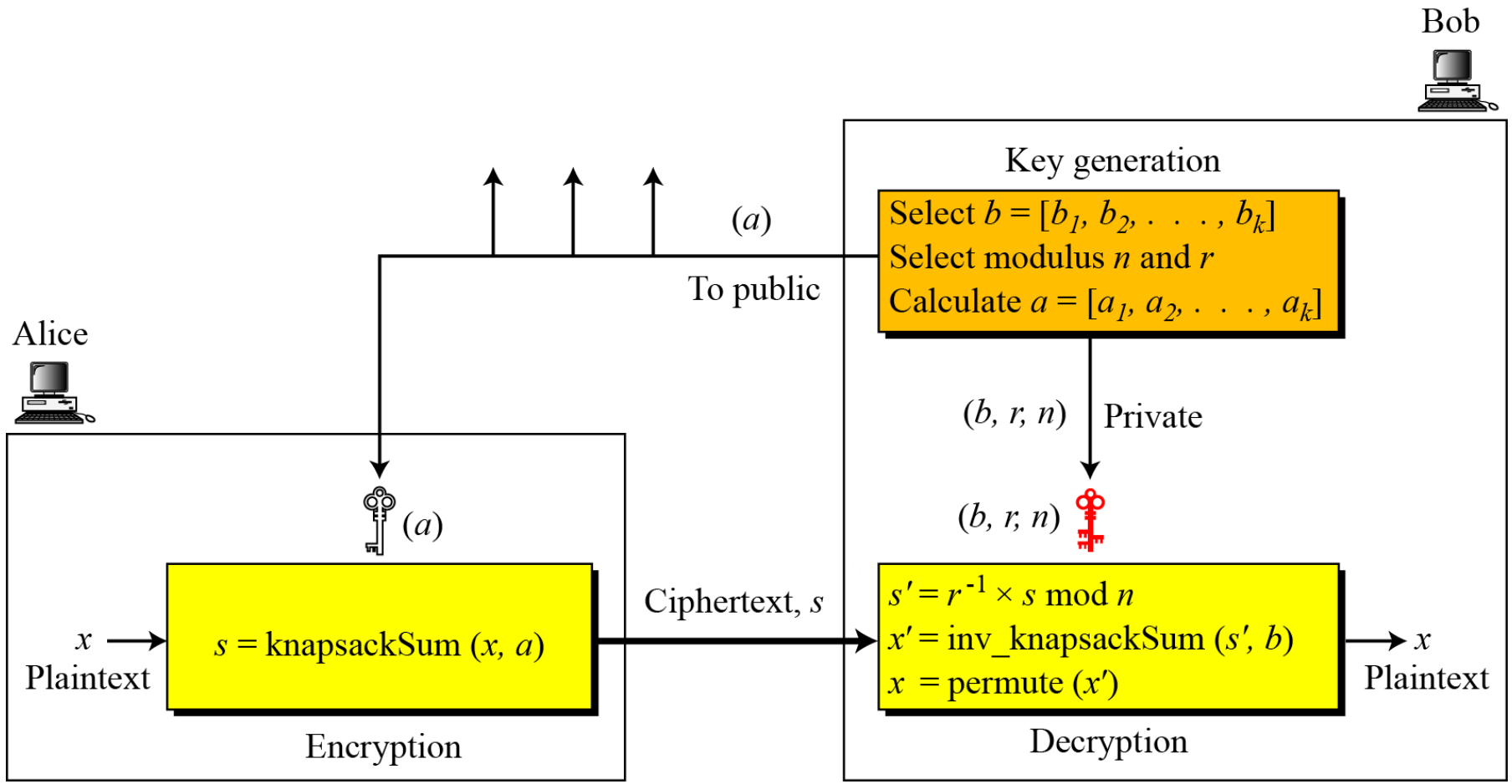
- **Example 1**: Given tuple $\mathbf{a} = [17,25,46,94,201,400]$ and $s = 272$. Find the tuple $\mathbf{x}$ such that $s = \mathbf{ax}$ using the inv_knapsackSum algorithm.

Intuitively , 272 = 201 + 46 + 25

i	a_i	s	$s \geq a_i$	x_i	$s = s - (a_i x_i)$
6	$a_6 = 400$	272	No	$x_6 = 0$	272
5	201	272	Yes	1	272 -201 = 71
4	94	71	No	0	71
3	46	71	Yes	1	71- 46 = 25
2	25	25	Yes	1	25-25 = 0
1	$a_1 = 17$	0	No	0	0

- The solution $\mathbf{x} = [0,1,1,0,1,0]$, and $\mathbf{a} = [25,46,201]$ are in the knapsack.
- 272 =25+46 + 201

KNAPSACK CRYPTOSYSTEM – OVERVIEW



KNAPSACK CRYPTOSYSTEM – KEY GENERATION

1. Create a super-increasing k-tuple $\mathbf{b} = [b_1, b_2, \dots, b_k]$.
2. Choose a modulus n, such that $n > b_1 + b_2 + \cdots + b_k$.
3. Select a random integer r that is relatively prime with n and $1 \leq r \leq n - 1$. gcd(r,n) = 1
4. Create a temporary k-tuple $\mathbf{t} = [t_1, t_2, \dots, t_k]$ such that
$$t_i = r \cdot b_i \pmod{n}$$
5. Select a permutation of k objects and find a new tuple $\mathbf{a} = \mathrm{Per}(\mathbf{t})$.
6. The public key is the $\mathbf{a}$. The private key is $(r, n, \mathbf{b})$.

KNAPSACK CRYPTOSYSTEM – ENCRYPTION

- Suppose Alice wishes to send a message to Bob:

1. Alice converts her message to a k-tuple $\mathbf{x} = [x_1, x_2, \ldots, x_k]$ in which x_1 is either 0 or 1. The tuple $\mathbf{x}$ is the **plaintext**.
2. Alice uses the knapsackSum algorithm to calculate s. She then sends the value of s as the **ciphertext**

KNAPSACK CRYPTOSYSTEM – DECRYPTION

1. Bob receives the ciphertext s.
2. Bob calculates $s' = r^{-1} \cdot s \pmod{n}$.
3. Bob uses inv_knapsackSum algorithm to create $\mathbf{x}'$.
4. Bob permutes $\mathbf{x}'$ to find $\mathbf{x}$. The **tuple** $\mathbf{x}$ is the recovered plaintext.

37⋆7 = 259, 37⋆11 = 407, 37⋆19 = 703,….,(37⋆313)mod900 = 781

KNAPSACK CRYPTOSYSTEM – EXAMPLE

➢ **Key Generation**:

1. Create a super-increasing k-tuple $\mathbf{b} = [7,11,19,39,79,157,313]$.
2. Choose a modulus $n = 900$.
3. Sum of k-tuple b = (7+11 + …+313) = S = 625 < n.
4. Select a random integer $r = 37$. checkgcd(37,900) = 1.
5. Create a temporary k-tuple $\mathbf{t} = [259,407,703,543,223,409,781]$ such that $t_i = 37 \cdot b_i \pmod{900}$
6. Modular inverse r^{-1}=73
7. The **public key** is the $\mathbf{a} = [259,407,703,543,223,409,781]$. The **private key** is $(37,900,[7,11,19,39,79,157,313])$.

INVERSE OF 37 IN Z_{900}

find inverse of 37 in Z_{900}.

1 = 37x + 900y

EEA

900 = 37(24) + 12 12 = 900- 37(24) eq 1

37 = 12(3) + 1 ; 1 = 37 - 12(3) eq 2

12 = 1(12) + 0.

gcd(37,900) = 1

1 = 37 - 12(3)

1 = 37 - [900 - 37(24)]3

1 = 37 - 3(900) + 72(37)

1 = 37(73) + 900(-3)

match with

1 = 37(x) + 900(y)

x = 73

inverse of 37 is 73

Proof

37*73 mod 900 = 2701 mod 900 = 1.

KNAPSACK CRYPTOSYSTEM – EXAMPLE

➢ Encryption:

1. Suppose Alice wishes to send the character 'g' to Bob. She first convert the character into a 7-bit ACSII code of $(1100111)_2$ and create the plaintext tuple $\mathbf{x} = [1,1,0,0,1,1,1]$.
2. Next, Alice computes the ciphertext $s = \text{knapsackSum}(\mathbf{a}, \mathbf{x})$ such that $s = 259 + 407 + 223 + 409 + 781 = \mathbf{2079}$

   public key is the $\mathbf{a} = [259,407,703,543,223,409,781]$
3. $\mathbf{x} = 1, 1, 0, 0, 1, 1, 1$
4. Finally, Alice send $s = 2079$ (cipher text) to Bob.

KNAPSACK CRYPTOSYSTEM – EXAMPLE

➢ Decryption:

1. Bob receives $s = 2079$ and computes
$$s' \equiv 2079 \cdot 37^{-1} \equiv 2079 \cdot 73 \equiv 567 \pmod{900}$$
2. Next, Bob computes $\mathbf{x}' = \text{inv\_knapsackSum}(s', b)$:
3. $\mathbf{b} = [7,11,19,39,79,157,313]$.

i	a_i	s	$s \geq a_i$	x_i'	$s = s - a_i$
7	a_7 =313	567	Yes	1	567-313= 254
6	157	254	Yes	1	254-157= 97
5	79	97	Yes	1	97-79= 18
4	39	18	No	0	18
3	19	18	No	0	18
2	11	18	Yes	1	18-11 = 7
1	7	7	Yes	1	7-7 = 0

KNAPSACK CRYPTOSYSTEM – EXAMPLE

3. The inverse knapsack algorithm returns $\mathbf{x}' = [1,1,0,0,1,1,1]$.
4. Bob computespermutation, $\mathbf{x} = \mathrm{Per}(\mathbf{x}') = [1,1,0,0,1,1,1]$
5. Finally, Bob revert the string $(1100111)_2 = g$ using ASCII code

KNAPSACK CRYPTOSYSTEM – SECURITY

- The underlying security behind the Knapsack Cryptosystem is based on the difficulty of solving the **Knapsack Problem.**

- Knapsack Problem is one of the **NP-Complete problems** that has NO feasible algorithm to solve it currently.

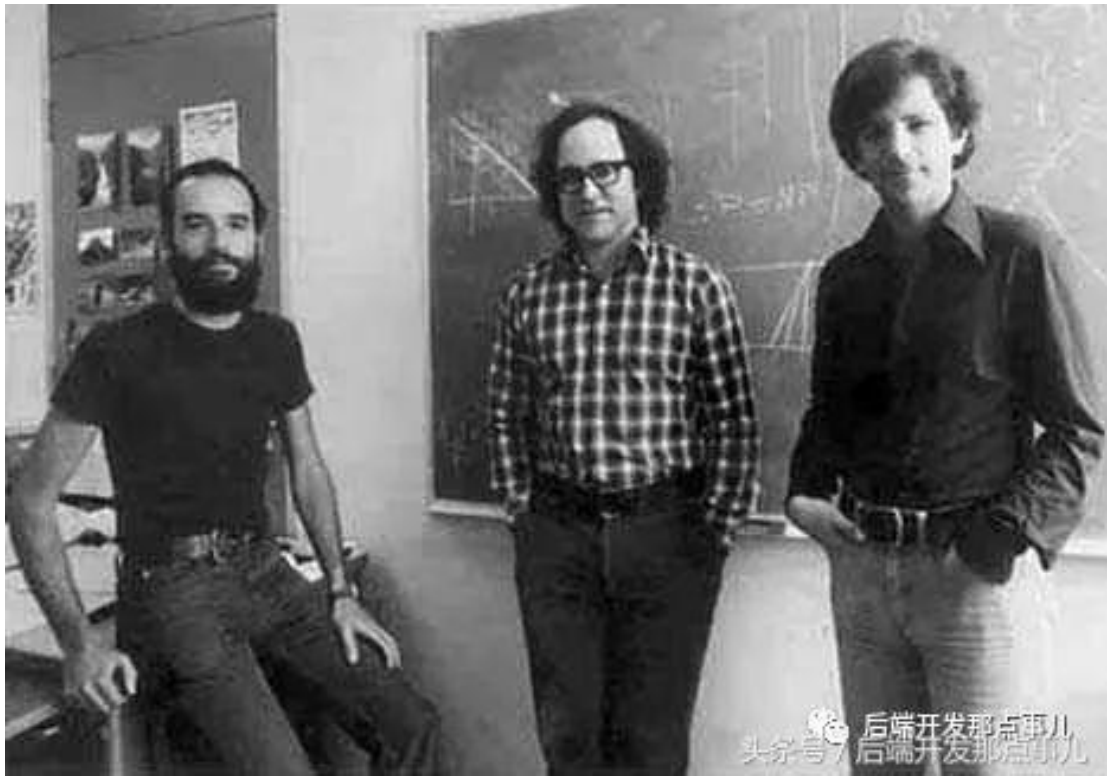
- Unfortunately, the Knapsack cryptosystem was **broken by Adi Shamir** in 1984. Hence,it is insecure and no longer be used.

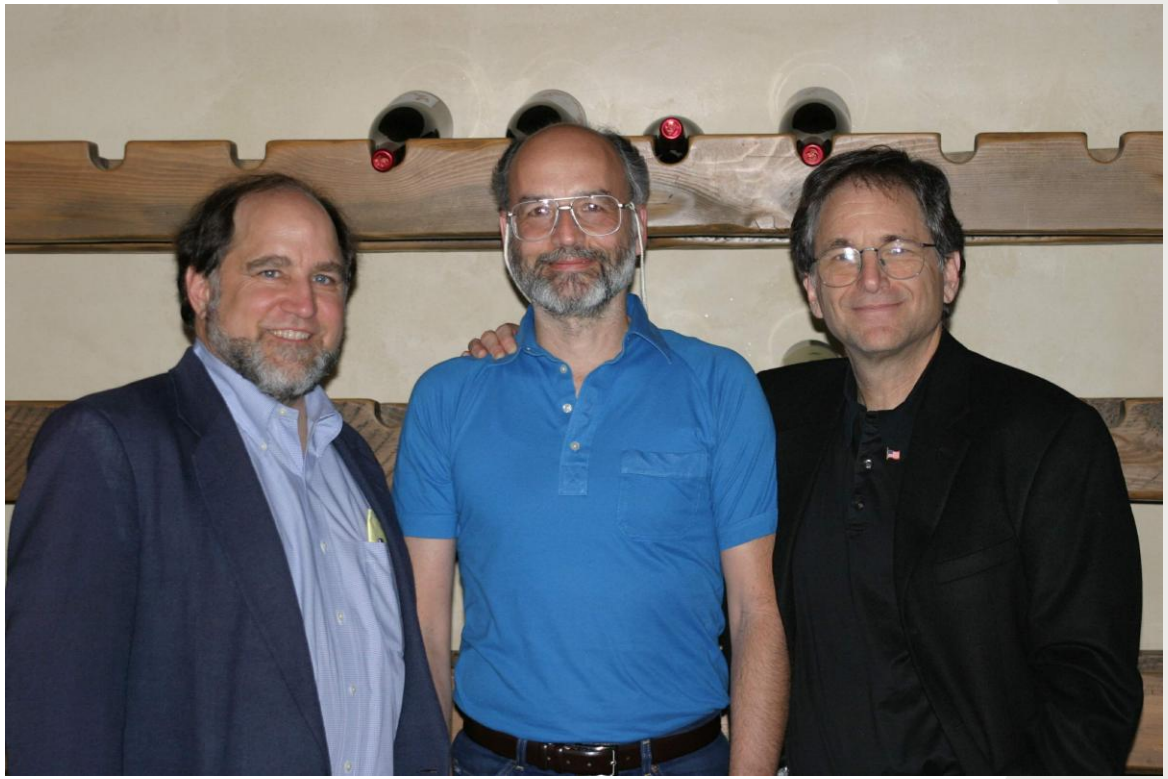
WHAT IS NP-COMPLETE ?

- NP-Complete problems
  - ✓There is NO efficient algorithm to solve the problem, but it has efficient algorithm to verify the solution.sha256 output.
  - ✓Any NP-Complete problems can be reduced into solving other NP-Complete problems In this case, **Knapsack Problem can be reduced into solving the Subset Sum Problem**.

- Knapsack Cryptosystem is based on Knapsack Problem, but why it is insecure in the end ?? 🤔🤔🤔
- Reason: The super-increasing property of the public tuple $\mathbf{a} = [a_1, a_2, \ldots, a_n]$.

RSA (RIVEST-SHAMIR-ADLEMAN) CRYPTOSYSTEM





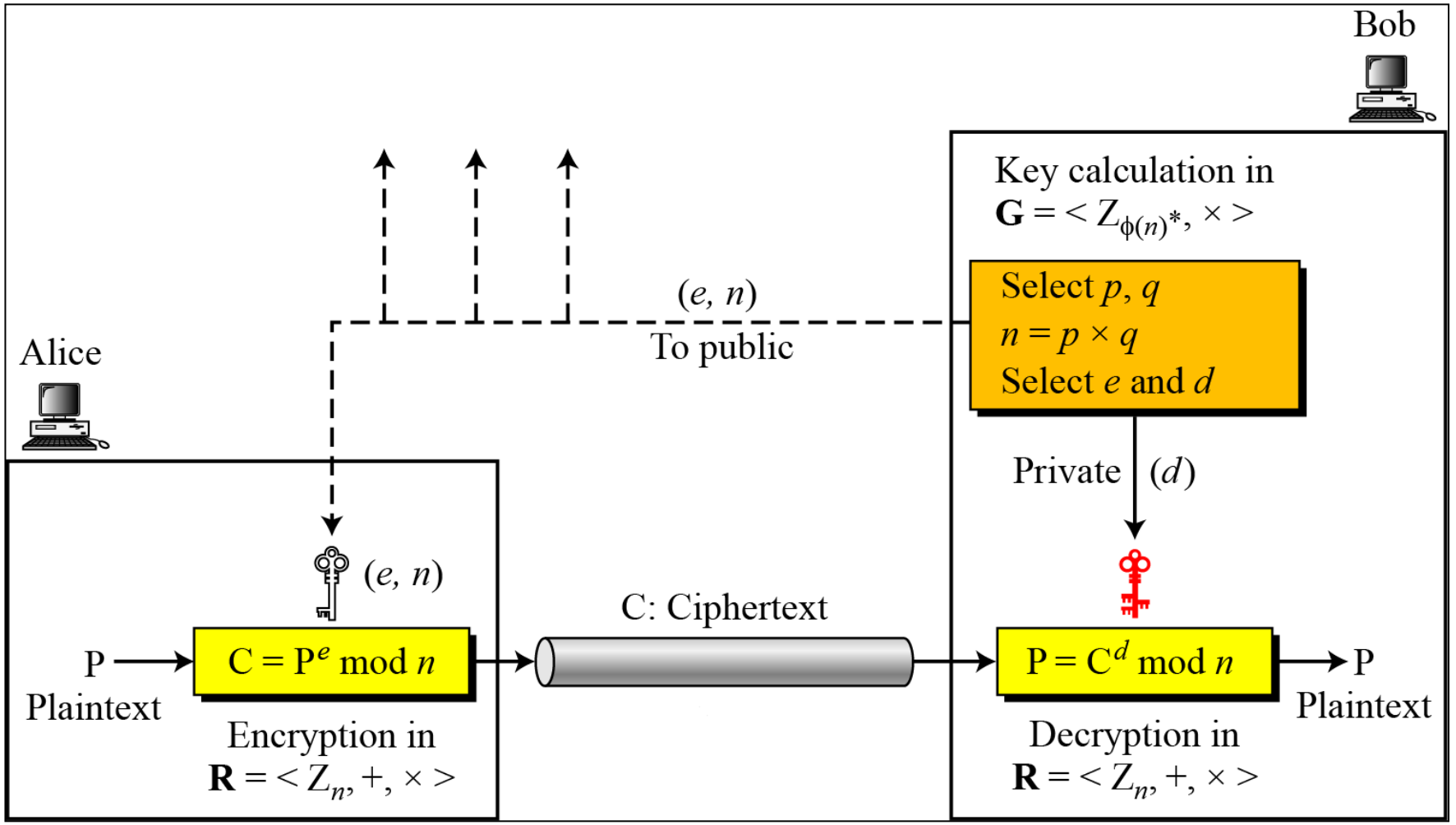
RSA CRYPTOSYSTEM

- **RSA cryptosystem** is the best known, and by far the most widely used public key encryption algorithm that was first published by Ron Rivest, Adi Shamir and Leonard Adleman of MIT in 1978.
- It is a **block cipher** in which the plaintext and ciphertext are integers between 0 and $2^n - 1$ for some security parameter n.
- The current **deployment standard** uses the size of $n = 2048$ (or 617 decimal digits). The value of $n < 2^{2048}$.
- The underlying security of RSA is based on the **hardness assumption of Integer Factorization Problem (IFP)** .

RSA CRYPTOSYSTEM – MATHEMATICS

- **Euler's Totient Function, $\phi(n)$**. This is required in computing the public-private keys.
- **Extended Euclidean Algorithm**. This is helpful in computing the multiplicative inverse of the public-private keys.
- **Fast Exponentiation Algorithm**. This is used to perform encryption and decryption using the public-private keys.
- **Euler's Theorem** This is to show the proof of correctness of the RSA cryptosystem.
- **Primality Testing**. This is used to generate large random prime numbers in the key generation.

RSA CRYPTOSYSTEM – OVERVIEW



RSA CRYPTOSYSTEM – KEY GENERATION

1. Selects two large primes p and q, computemodulus $n = pq$.
2. Computesthe Euler Totient $\phi(n) = (p-1)(q-1)$.
3. Selects a random public exponent e such that $\gcd(e, \phi(n)) = 1$, and $1 < e < \phi(n)$.
4. Computesthe private exponent d such that $ed \equiv 1 \pmod{\phi(n)}$.
5. Publish the public keys (e, n) and keeps the private key (d).

❖ The recommendedsize of RSA is 2048 bits (the modulus n), this implies that the selected primes p and q are each of 1024 bits.

RSA CRYPTOSYSTEM – ENCRYPTION & DECRYPTION

- Suppose Alice wishes to send plaintext $P < n$ to Bob.

1. She obtains Bob's public keys (e, n) and computes ciphertext $C \equiv P^e \pmod{n}$.
2. She sends the ciphertext C to Bob.

- When Bob receivesthe ciphertext C from Alice.

1. He computes $P \equiv C^d \pmod{n}$ using his private key (d).

$(P^{encrypt})^{decrypt}$

RSA CRYPTOSYSTEM – ENCRYPTION & DECRYPTION

- Why is the encryption and decryption possible in RSA cryptosystem?

✓ **Proof of correctness**

$$
\begin{aligned}
C^d \pmod n &\equiv (P^e)^d \pmod n && ; C \equiv P^e \pmod n \\
&\equiv P^{ed} \pmod n \\
&\equiv P^{1\, mod(\phi(n))} \pmod n && \star \text{Since } ed \equiv 1 \pmod{\phi(n)} \\
&\equiv P^1 \cdot P^{\phi(n)} \pmod n \\
&\equiv P \cdot 1 \pmod n && \star \text{Since } P^{\phi(n)} \equiv 1 \pmod{\phi(n)} \\
&\equiv P \pmod n
\end{aligned}
$$

RSA CRYPTOSYSTEM – TRAPDOOR ONE-WAY FUNCTION

- The basis of RSA cryptosystem lies on the **one-way function** of the encryption equation

$$C \equiv P^e \pmod{n}$$

- The forward encryption is **computationally feasible** given the public keys (e, n).
- However, given only ciphertext C and public key e, inverting the plaintext $P \equiv C^d \pmod{n}$ is computationally infeasible **if the modulus n is sufficiently large**.
- **Trapdoor**: **the private key d**. With this value, it is computationally feasible to decrypt the ciphertext and recover the plaintext.

RSA CRYPTOSYSTEM – SMALL EXAMPLE

- **Key Generation**:

1. Selects two primes $p = 11$ and $q = 17$, compute modulus $n = (11)(17) = 187$.
2. Computes the Euler Totient $\phi(187) = (11 - 1)(17 - 1) = 160$.
3. Selects a random public exponent e such that $\gcd(e, \phi(n)) = 1$, and $1 < e < \phi(n)$. Here, we choose $e = 7$.
4. Computes the private exponent d such that $7d \equiv 1 \pmod{160}$. Using the Extended Euclidean Algorithm, $d \equiv 23 \pmod{160}$.
5. Publish the public keys $(7,187)$ and keeps the private key (23).

RSA CRYPTOSYSTEM – SMALL EXAMPLE

- **Encryption**:

1. Suppose Alice wants to send plaintext $P = 88$ to Bob. She obtains the public keys $(7,187)$ and computes $C \equiv 88^7 \pmod{187} \equiv 11 \pmod{187}$.
2. She sends the ciphertext $C \equiv 11 \pmod{187}$ to Bob.

- **Decryption**:

1. Bob when receives Alice's ciphertext $C \equiv 11 \pmod{187}$, computes $P \equiv 11^{23} \pmod{187} \equiv 88 \pmod{187}$ and recover the plaintext $P \equiv 88 \pmod{187}$.

- $C \equiv 88^7 \pmod{187}$; $P \equiv 11^{23} \pmod{187} \equiv 88 \pmod{187}$

RSA CRYPTOSYSTEM – SECURITY

- **Security** behind RSA Cryptosystem:
  1. (Strongest) Integer Factorization Problem (IFP): Given n, find its prime factors of p and q.
  2. e^{th}-root problem: Given $C \equiv P^e \pmod{n}$, find $P \equiv \sqrt[e]{C} \pmod{n}$.
  3. RSA key-equation: Given $ed \equiv 1 \pmod{\phi(n)}$, find k, d such that
  $$ed + k\phi(n) = 1.$$

- IFP is one of the **NP problems** but NOT **NP-Complete**
  - NOT NP-Complete as IFP has no reduction into other NP-Complete problems

INTEGER FACTORIZATION PROBLEM (IFP):

- n = 17,408,604,539 = pq
- $\phi(n)$= 17,408,340,120 = (p-1)(q-1)
- Find p & q
- Can you find $\sqrt[168]{C}$
- $ed \equiv 1 \pmod{\phi(n)}, e = 7, n = 187, , \phi(n) = 160$
- Find k and d, where $ed + k\phi(n) = 1$.

RSA CRYPTOSYSTEM – SECURITY

❖ **Why is IFP difficult?**

- IFP is a **one-way function**, given two primes p and q, you can compute $n = pq$ efficiently and easily. Example: given $p = 7841$ and $q = 7621$, we can compute $n = 59{,}756{,}261$
- Now, given $n = 45{,}605{,}689$, **factor it into two primes**.
- **The answer**: $p = 6599$ and $q = 6911$.

- **Question:** If IFP is difficult, **why not we use more primes** in constructing n? That is, why not $n = pqrstuv$, for example? 🤔🤔🤔

RSA CRYPTOSYSTEM – CRYPTANALYSIS & ATTACKS

❖There are generally five possible approaches to attacking RSA:

1. Brute Force
2. Mathematical Attacks
3. Timing Attacks
4. Hardware Fault-Based Attacks
5. Chosen Ciphertext Attacks (CCA)

RSA CRYPTOSYSTEM – CRYPTANALYSIS & ATTACKS

1. Brute Force

- This attack involves **trying all the possible private keys** in attempt to decrypt the intercepted ciphertext successfully.
- This attack is impractical if the security parameter n chosen is sufficiently large.
- For example, current standard RSA deployment uses $n = 2048$, this implies that the private key is at least the size of 1024-bit, which is impractical to brute force it efficiently.

RSA CRYPTOSYSTEM – CRYPTANALYSIS & ATTACKS

2. Mathematical Attacks.

- This strategy cryptanalyzes the mathematics and the mathematical structures utilized in designing the RSA cryptosystem, primarily the **Integer Factorization Problem (IFP)**. $d: priv\ key, e: pubkey$
- There three possible approaches to cryptanalyzing RSA mathematically:
  a. Factor n into its two prime factors. This enables calculation of $\phi(n)$, which next enables determination of $d \equiv e^{-1} \pmod{\phi(n)}$.
  b. Determine $\phi(n)$ directly without first finding p and q. Again, this enables determination of $d \equiv e^{-1} \pmod{\phi(n)}$.
  c. Determine d directly without first finding $\phi(n)$.

RSA CRYPTOSYSTEM – CRYPTANALYSIS & ATTACKS

3. Timing Attacks.

- A cryptographic consultant Paul Kocher, demonstrated that a snooper can determine a private key d by keeping track of **how long a computer takes to decrypt messages**, hence the name of timing attacks.
- These attacks are applicable not only to RSA but to other public-key cryptosystems, and they are alarming for two reasons:
  a. They come from a completely unexpected direction (**side-channel**).
  b. They are the **ciphertext-only attack**.

- $C \equiv 88^7 \pmod{187}; P \equiv 11^{23} \pmod{187} \equiv 88 \pmod{187}$

If exponent is small,
Time is small.

RSA CRYPTOSYSTEM – CRYPTANALYSIS & ATTACKS

- To prevent such timing attacks, some countermeasures can be taken:
  a. **Constant exponentiation time.** Ensures that all exponentiations take the same amount of time before returning a result; this is a simple fix, but it degrades the performance
  b. **Random delay.** Better performance could be achieved by adding a random delay to the exponentiation algorithm to confuse the timing attack.
  c. **Blinding.** Multiply the ciphertext by a random number before performing an exponentiation. This process prevents the attacker from knowing what ciphertext bits are being processed inside the computer and therefore prevents the bit-by-bit analysis essential to the timing attack.

RSA CRYPTOSYSTEM – CRYPTANALYSIS & ATTACKS

4. Hardware Fault-Based Attacks.

- An attack on a processor that is generating RSA digital signatures.
  - It induces **faults in the signature computation** by **reducing the power to the processor**.
  - The faults cause the software to produce invalid signatures which can then be analyzed by the attacker to recover the private key.
  - The attack algorithm involves **inducing single-bit errors** and observing the results.
- While worthy of consideration, this attack does not appear to be a serious threat to RSA since it requires that the attacker to **have physical access** to the target machine and is able to directly control the input power to the processor.

RSA CRYPTOSYSTEM – CRYPTANALYSIS & ATTACKS

5. Chosen Ciphertext Attack (CCA).

- The strongest cryptanalysis environment where an attacker has the full ability to simulate the ciphertexts of his choice and is then given the corresponding decrypted plaintexts.
- The adversary exploits properties of RSA and selects blocks of data that, when processed using the target's private key, yield information needed for cryptanalysis.
- To counter such attacks, RSA Security Inc. recommends modifying the plaintext using a procedure known as optimal asymmetric encryption padding (OAEP).

How U encrypt ? U use public key (known to everyone).
Encrypt anything with any text, U will get the cipher text for your plaintext.

RSA OPTIMAL ASYMMETRIC ENCRYPTION PADDING (RSA/OAEP)

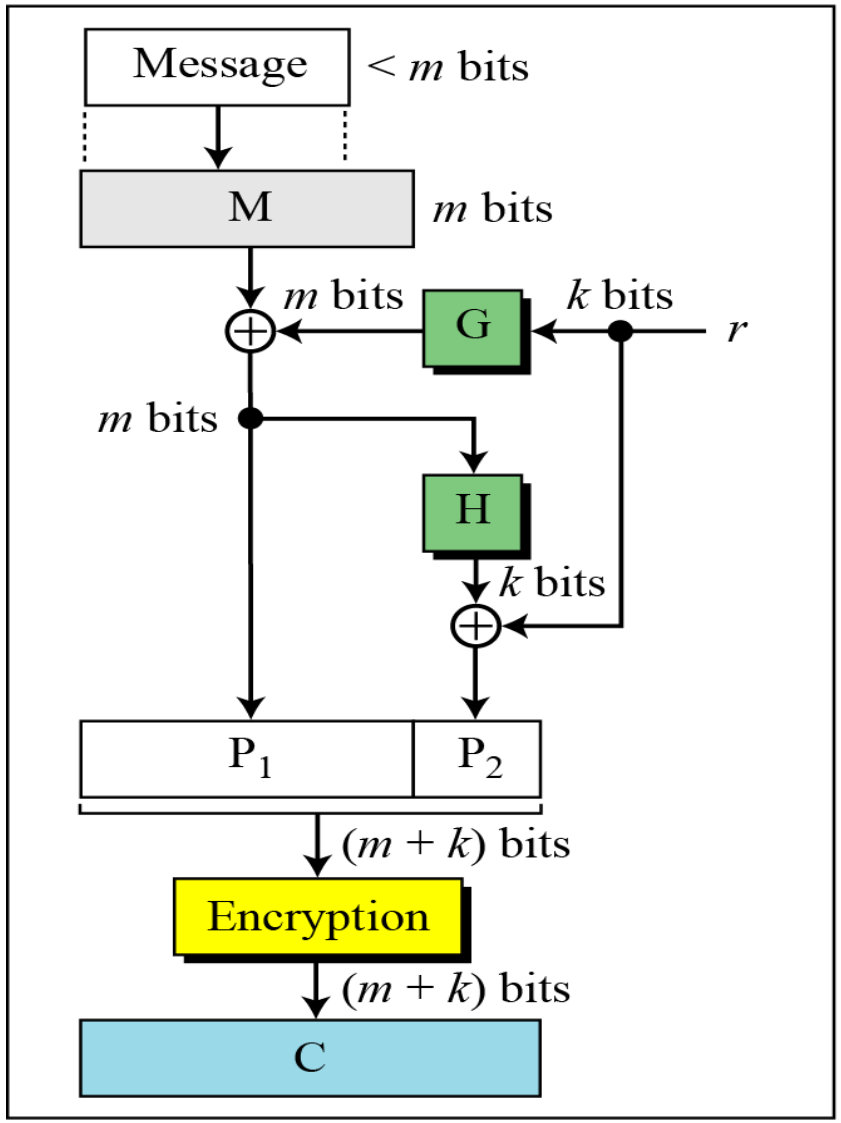
- The **Optimal Asymmetric Encryption Padding (OAEP)** is a **padding scheme** that was introduced by Bellare and Rogaway in 1995.
- It is a **Feistel structure** which uses a pair of random oracles G and H prior to asymmetric encryption, and is often used with RSA, hence the name RSA/OAEP.
- It **features two main goals** in asymmetric encryption:

1. It adds **randomness** and convert a deterministic encryption into **probabilistic encryption**
2. It resolves the CCA insecurity issue via preventing partial ciphertext decryption attacks.

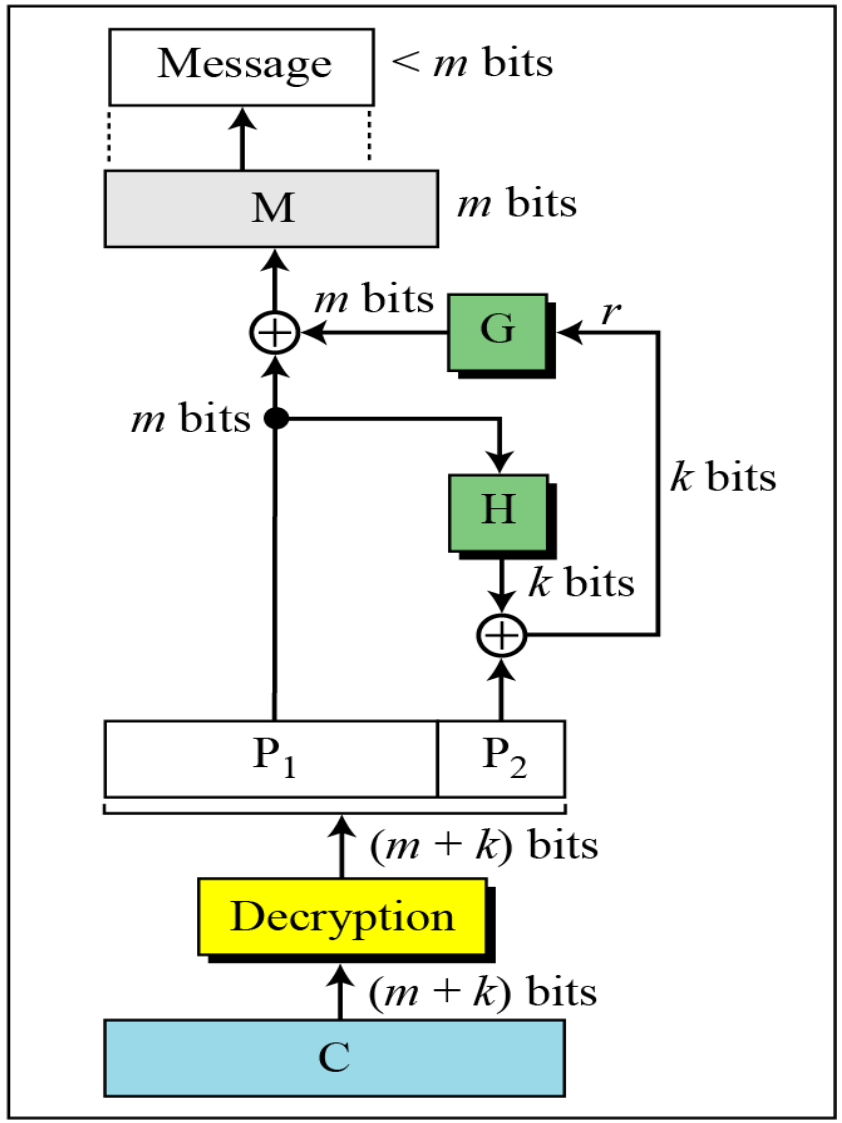
M: Padded message
r: One-time random number

P: Plaintext ($P_1 \parallel P_2$)
C: Ciphertext

G: Public function (k-bit to m-bit)
H: Public function (m-bit to k-bit)



Sender



Receiver

ELGAMAL
CRYPTOSYSTEM
(OPTIONAL)

ELGAMAL CRYPTOSYSTEM

- A public-key cryptosystem designed by Taher ElGamal in 1985 based on the **Discrete Logarithm Problem (DLP)** and the **Diffie-Hellman Key Exchange (DHKE)**.
- The cryptosystem that uses the DHKE mechanism in establishing a shared secret key, which is next used to perform decryption and recover the plaintext message.
- Current security strength of DLP lies on the prime p of order 1024-bit.

Public Parameter Generation	
A Trusted Third Party (TTP) chooses and publishes a large prime p and an element $g \pmod p$ of large prime order.	
Alice	**Bob**
Public Key Generation	
1. Chooses private key $1 \leq a \leq p-1$. 2. Computes $A = g^a \pmod p$. 3. Publishes her public key A.	
Encryption	
	1. Chooses message M and an ephemeral key k. 2. Computes ciphertext $C_1 \equiv g^k \ (mod\ p)$, and $C_2 \equiv M \cdot A^k \pmod p$ 3. Send ciphertext tuple (C_1, C_2).
Decryption	
1. Compute message $M \equiv (C_1^a)^{-1} \cdot C_2 \pmod p$	

ELGAMAL CRYPTOSYSTEM

❖Proof of correctness

$$
\begin{aligned}
(C_1^a)^{-1} \cdot C_2 &\equiv \left(g^{ka}\right)^{-1} \cdot \left(M \cdot A^k\right) && (\mathrm{mod}\ p) \\
&\equiv \left(g^{ak}\right)^{-1} \cdot \left(M \cdot A^k\right) && (\mathrm{mod}\ p) \\
&\equiv \left(A^k\right)^{-1} \cdot \left(M \cdot A^k\right) && (\mathrm{mod}\ p) \\
&\equiv A^{-k} \cdot \left(M \cdot A^k\right) && (\mathrm{mod}\ p) \\
&\equiv M && (\mathrm{mod}\ p)
\end{aligned}
$$

MAIN REFERENCES FOR CHAPTER 9

1. Forouzan, B.A. "***Introduction to Cryptography and Network Security***", Third Edition, McGraw-Hill, 2008.
2. Stallings, W. "***Cryptography and Network Security: Principles and Practices***", Fifth Edition, Pearson Prentice Hall, 2011.

~ THE END ☺ ~